

LLM-Assisted Big Data Integration

Silvia Tommaso, Francesco Pittacolo

Repository link: <https://github.com/silviatommaso/BigData-Integration>

Introduction

Data integration is the process of combining data originating from multiple heterogeneous sources in order to provide a unified, comprehensive, and actionable view of the available information. This process involves addressing several challenges, including poor data quality, incompatible data formats, and fragmented data sources.

In this context, Big Data introduces additional challenges due to its characteristic volume, variety, and velocity.

The objective of this project is to design, implement, and evaluate a traditional data integration pipeline and to compare its performance with a pipeline that incorporates Large Language Models into selected stages of the integration process. The study focuses on the following three fundamental phases:

- **Schema Alignment:** mapping source attributes onto a mediated schema and identifying correspondences between elements that represent the same concept across different data sources.
- **Record Linkage:** identifying records that refer to the same real-world entity.
- **Data Fusion:** reconciling conflicting values and producing a single integrated representation for each entity.

1 Methodology

This section describes the methodology adopted to integrate heterogeneous movie datasets into a unified representation. The proposed pipeline investigates both a traditional algorithmic approach and an LLM-assisted solution. The datasets, the preprocessing operations applied prior to the integration pipeline, and the experimental configuration are presented in Section 2.

1.1 Schema Alignment

Although heterogeneous datasets may describe the same domain, they can differ in schema structure and attribute naming conventions. Schema alignment is therefore required to identify semantic correspondences between attributes and define a common representation. Before performing the alignment, minor normalization operations, including lowercase conversion, removal of special characters, and index reorganization, were applied to reduce inconsistencies caused by formatting differences.

1.1.1 Traditional Baseline

The baseline schema alignment approach relied on a profile-based attribute matching, which rather than considering only attribute names, combines schema-level information with statistical and structural characteristics extracted directly from the attribute values.

Column Profiling For each column, a set of descriptive features was extracted to obtain a compact representation of the attribute, as shown in Figure 1.

```
column_profile = {
    "avg_length": float(row_length.mean()),
    "avg_words": float(word_count.mean()),
    "numeric_percentage": float(numeric_percentage),
    "cardinality": float(values_cardinality),
    "is_purely_numeric": False,
    "avg": 0.0,
    "min": 0.0,
    "max": 0.0
}
```

Fig. 1: Column profile structure

Since declared data types were not always reliable or sufficiently informative, attribute types were inferred directly from the data. Attributes with a `numeric_percentage` higher than 70% were classified as numerical. For these attributes, additional statistical descriptors were computed, namely `min`, `max`, and `avg`, which were used to capture their numerical distribution and enable cross-dataset comparison.

Within the same profiling framework, **cardinality** was defined as the ratio between the number of distinct values and the total number of records in a column. It was used to characterize the semantic role of an attribute: *low* cardinality typically indicates categorical attributes (e.g., movie genres), while *high* cardinality is associated with descriptive or identifying attributes (e.g., movie titles).

Similarity Computation The extracted features were then combined into two weighted similarity measures, namely a structural and numerical similarity score:

$$S_{structure} = w_1 \cdot S_{length} + w_2 \cdot S_{words} + w_3 \cdot S_{num_density} + w_4 \cdot S_{cardinality} \quad (1)$$

$$S_{numeric} = v_1 \cdot S_{range} + v_2 \cdot S_{avg} \quad (2)$$

where $w_i, v_j \in [0,1]$ are weighting coefficients controlling each component contribution

The numerical similarity score is computed only when both attributes are classified as numerical. Consequently, the data-level similarity is defined as:

$$S_{data} = \begin{cases} \alpha \cdot S_{structure} + (1 - \alpha) \cdot S_{numeric} & \text{if both attributes are numerical} \\ S_{structure} & \text{otherwise} \end{cases} \quad (3)$$

where $\alpha \in [0,1]$ is a weighting coefficient controlling the contribution of structural and numerical components.

Finally, after computing the syntactic similarity score S_{name} between attribute names, data-level and name-based similarities are combined to produce the final matching score:

$$S_{final} = \beta \cdot S_{data} + (1 - \beta) \cdot S_{name} \quad (4)$$

where $\beta \in [0,1]$ is a weighting coefficient controlling the relative importance of data-level and name-based similarity.

Attribute Clustering For each attribute, an initial cluster is instantiated. Given a list of pairwise matching scores, all correspondences are sorted in descending order of similarity, ensuring that the most reliable matches are processed first. Clustering is then performed in a greedy manner, progressively constructing groups of aligned attributes.

Given a predefined threshold τ , only pairs of attributes (a_i, a_j) satisfying:

$$S_{final}(a_i, a_j) \geq \tau \quad (5)$$

are considered for clustering. For each such pair, three cases are considered: (i) neither a_i nor a_j has been assigned to a cluster, in which case a new cluster is created; (ii) exactly one

of the two attributes already belongs to a cluster, in which case the other is added if it is not already assigned; (iii) both attributes belong to different clusters, in which case a merge is attempted.

A structural constraint is enforced during all operations: each cluster can contain at most one attribute per dataset. Formally, for any cluster C_k :

$$\forall a_p, a_q \in C_k, \quad D_p \neq D_q \quad (6)$$

If merging two clusters would violate this constraint, the operation is discarded. This ensures that each cluster represents a cross-source semantic alignment rather than intra-source grouping.

Finally, the clusters are transformed into a tabular schema, where each row corresponds to a set of aligned attributes and each column represents a source dataset. Figure 2 provides an illustrative example of the alignment table.

```
imdb_3,rotten_tomatoes,imdb_5,roger_ebert
cast,cast,actors,actors
contentrating,contentrating,,critic_rating
creators,creators,,
director,director,directors,directors
year,year,year,year
,,pg_rating
,summary,,
...
```

Fig. 2: Matching attributes table example

The final schema is derived by removing rows containing null values and selecting the attributes from the first column, which is used as the reference representation of the unified schema.

1.1.2 LLM-Assisted Schema Alignment

Alternatively, an LLM-based approach was employed to perform schema matching in a data-driven manner, querying multiple LLMs to improve robustness and assess model variability.

The models were accessed via the Groq API, and two prompting configurations were considered based on a shared set of instructions defining the schema alignment task: the first relied exclusively on representative samples extracted from each dataset, while the second additionally included attribute-level descriptions.

Sample selection favored records with few missing values and a sufficient number of differing attributes, providing heterogeneous and informative examples of the underlying data. The inclusion of attribute descriptions improves alignment accuracy and reduces the risk of

incorrect matches.

The prompt also included an example of the expected output format, namely a CSV table whose structure is consistent with the one shown in Figure 2.

Each model’s output, latency, and token usage were recorded for analysis. The model that achieved the best performance in terms of alignment accuracy and token usage was then selected as the final predictor.

1.2 Record Linkage

After schema alignment, semantically equivalent attributes from different datasets become comparable. However, records describing the same real-world entity may still appear as distinct tuples across sources. Therefore, a record linkage phase was required to identify records referring to the same movie and establish correspondences among them.

1.2.1 Canopy-Based Candidate Generation

Performing pairwise comparisons between all records across datasets may become computationally expensive, as the number of candidate pairs grows quadratically with the dataset size. To address this issue, a blocking phase is introduced to reduce the search space by restricting comparisons to smaller candidate sets.

Instead of using a strict blocking strategy based on exact matching rules, the Canopy Clustering algorithm was adopted to partition records into overlapping candidate sets (canopies). Each canopy is constructed around a representative center record, built upon a subset of schema attributes referred to as *blocking keys*, and expanded by including similar instances according to a two-threshold similarity scheme.

TF-IDF Matrix Construction Instead of relying on exact token matching, each blocking key attribute is represented using a character-level TF-IDF encoding (e.g., bi-grams and tri-grams). This representation allows the model to capture partial similarities between strings while mitigating the effects of a highly heterogeneous dataset, which can lead to a loss of matching opportunities.

```
vectorizer_blocking_key = TfidfVectorizer(
    analyzer="char_wb",
    ngram_range=(x, y),
    min_df=z
)
```

Fig. 3: Example configuration of the TF-IDF vectorizer used for blocking key representation

The parameters x and y in Figure 3 define the range of character n-grams used during vectorization. For instance, setting $x = 2$ and $y = 4$ corresponds to using bi-grams up to quadri-grams.

The parameter *min_df* specifies the minimum document frequency threshold and is used to filter out rare patterns, thereby reducing noise in the representation.

Once vectorized, record similarity is computed using cosine similarity over the TF-IDF representations.

Clustering The clustering procedure is based on a dual-threshold strategy. A record is selected as the center of a canopy, and its similarity with the remaining records is computed. A first, looser threshold (T_1) determines whether a record is sufficiently similar to be included in the canopy. A second, stricter threshold (T_2), identifies whether a record is so similar to the center that it can therefore be removed from the candidate pool, preventing them from being selected as future canopy centers.

When both records contain values for all blocking keys, the inclusion criterion requires that the corresponding similarity constraints are simultaneously satisfied. If one or more blocking keys are missing, the comparison is performed only on the available attributes, thus preserving robustness to missing values, while avoiding the exclusion of potentially matching records.

The process is repeated iteratively until all records are assigned to at least one canopy, resulting in a set of partially overlapping clusters used as candidate blocks for the subsequent record linkage phase.

1.2.2 Record Matching

After the blocking phase, record matching is performed to determine whether candidate record pairs refer to the same real-world entity.

The matching process is designed to be schema-independent: the attributes used for comparison, their relative importance, and the similarity functions applied to their values are defined through a configurable matching strategy. This allows the system to adapt to different dataset schemas without requiring changes to the matching algorithm itself.

For each candidate pair (r_i, r_j) , the system computes an overall similarity score by combining multiple attribute-level similarities. The record-level similarity is defined as:

$$S_{record}(r_i, r_j) = \sum_{k=1}^n w_k S_k(r_i, r_j) \quad (7)$$

where S_k represents the similarity computed for attribute k and w_k represents the corresponding importance weight. The weights are defined according to the characteristics of the considered attributes and their expected contribution to the matching decision.

Although the matching procedure is independent from the specific schema, the similarity computation is attribute-dependent, since different comparison strategies can be applied according to the representation and semantic nature of the values. For atomic textual attributes, character-based similarity functions can be used, while numerical attributes can be evaluated through rule-based comparison strategies.

For multi-valued attributes, values are represented as sets and compared using set-based similarity measures, such as the Jaccard similarity coefficient:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|} \quad (8)$$

where A and B represent the sets of values extracted from the two records.

When attributes may contain heterogeneous representations, such as either single values or lists of values, a hybrid strategy can be adopted by combining different similarity functions and selecting the most appropriate comparison according to the value representation. This allows the framework to handle different attribute formats without modifying the overall matching procedure.

A candidate pair is classified as a match if:

$$S_{record}(r_i, r_j) \geq \theta \quad (9)$$

where θ is the decision threshold controlling the acceptance of a candidate correspondence. For each accepted match, the system stores the identifiers of the two records, the global similarity score, and the individual contribution of each attribute similarity. The resulting matches are then used in the subsequent entity clustering phase.

1.2.3 LLM-assisted Record Matching

To improve the resolution of ambiguous cases, an LLM-based decision layer can be integrated on top of the similarity-based matching process. The LLM is not used as a replacement for the traditional similarity computation, but as an additional semantic evaluation step for uncertain candidate pairs.

First, the record similarity function described in the previous section is applied using a relaxed candidate threshold θ' , where:

$$\theta' < \theta \quad (10)$$

Candidate pairs with a similarity score below θ' are discarded. The remaining pairs are divided into two categories. Pairs with high similarity are directly accepted as matches if:

$$S_{record}(r_i, r_j) \geq \theta \quad (11)$$

while pairs satisfying:

$$\theta' \leq S_{record}(r_i, r_j) < \theta \quad (12)$$

are considered ambiguous cases and are evaluated by the LLM.

For each ambiguous pair, the LLM receives the attribute values of the two records and returns a structured response in JSON format containing a match indicator, a confidence score, and an explanation. A candidate evaluated by the LLM is accepted only if the model classifies it as a match and the confidence score is above a predefined threshold γ :

$$match = 1 \wedge confidence \geq \gamma \quad (13)$$

This hybrid strategy combines the efficiency of similarity-based approaches with the semantic reasoning capabilities of language models, limiting LLM usage only to cases where traditional similarity measures provide insufficient evidence.

To reduce redundant requests, each LLM evaluation is associated with a unique hash-based identifier generated from the tuple (`id1`, `id2`, `classical_score`), where `id1` and `id2` represent the identifiers of the two compared records, and `classical_score` denotes the similarity score produced by the rule-based matching algorithm.

The generated identifier, together with the corresponding LLM response, is stored in a CSV file that serves as a persistent cache. Before issuing a new request, the system checks this cache for an existing entry with the same identifier. If a match is found, the cached response is reused; otherwise, a new request is sent to the LLM and the resulting response is appended to the CSV file for future use.

1.2.4 Entity Clustering

After the record matching phase, the resulting matches are grouped into entities using a graph-based clustering approach. An undirected graph is constructed in which each record involved in at least one match corresponds to a node, while each matched pair defines an edge connecting the corresponding records. The similarity score associated with each match is also stored as the edge weight. Although the edge weights are not used by the current connected-components clustering procedure, they preserve information about the strength of each match and can support future analyses or alternative clustering strategies.

Entities are identified by computing the connected components of the graph. Each connected component represents a group of records that are directly or indirectly linked through matching relationships and is therefore considered to correspond to a single real-world movie entity.

This approach naturally captures transitive relationships. For example, if record *A* matches record *B* and record *B* matches record *C*, then all three records are assigned to the same entity, even if records *A* and *C* were not directly matched. Consequently, the method remains effective even when some pairwise matches are missing, provided that the records are still linked through intermediate matches.

Each connected component is assigned a unique entity identifier. The records belonging to each entity are combined with their original attributes and stored in the output dataset. Finally, records that do not appear in any matched pair are identified as singleton entities by comparing the identifiers contained in the graph with those in the original dataset. These records are exported to a separate CSV file for further analysis.

1.3 Data Fusion

After the record linkage phase, each cluster contains multiple records referring to the same real-world entity but originating from different data sources. The data fusion step is then applied to generate a single consolidated representation of each entity by resolving conflicts among attribute values and aggregating information across sources.

The fusion strategy distinguishes between atomic attributes, for which a single representative value is selected, and multi-valued attributes, for which a confidence-based aggregation

mechanism is applied.

Atomic fields For attributes containing a single value per record, value conflicts across sources are resolved using a weighted voting scheme. Each source contributes to the final decision according to a predefined reliability weight. The final value is selected as the one with the highest aggregated score across all sources.

Multivalued fields For attributes containing a set of values rather than a single atomic entry, value fusion is performed through a confidence-based aggregation strategy that accounts for both the frequency and reliability of the sources.

Each candidate value is assigned a confidence score computed as the sum of the weights of the sources in which it appears. A dynamic threshold, defined as a fraction of the maximum observed score, is then applied to filter out low-confidence values.

```
max_score = max(score.values())
dynamic_threshold = max_score * threshold_ratio
```

Fig. 4: Confidence filtering procedure

To avoid redundancy, a secondary deduplication step is applied to the remaining candidates. String similarity and token overlap are jointly used to identify semantically equivalent values, accounting for variations in word order and lexical form. The decision is guided by a dynamic similarity threshold that adapts to the length of the candidate strings.

```
if actual_sim >= similarity_threshold or (words_sharing and contained):
    duplicate_found = True
```

Fig. 5: Duplicate filtering procedure

This process produces a unified representation of each attribute at cluster level, ensuring consistency and reducing redundancy across heterogeneous data sources.

1.4 Final Dataset Construction

After the data fusion phase, the final dataset is obtained by merging fused entities with singleton records that were not assigned to any cluster during the record linkage process. Before merging, singleton records are normalized to ensure schema compatibility with fused entities. This includes aligning attribute names and formats and mapping source identifiers into the same representation used in clustered records. Missing attributes are retained as null values when no reliable information is available.

This step results in a unified dataset that integrates both consolidated multi-source entities and non-merged records in a consistent schema.

2 Experimental Protocol

This section describes the experimental setup adopted to evaluate both the baseline and LLM-assisted pipelines, conducted using four heterogeneous datasets derived from the Magellan project, differing in terms of origin, quality and schema structure.

Datasets The datasets originate from movie-related platforms and were collected through dedicated APIs or web scraping techniques. The three data sources considered are **Roger Ebert**, **Rotten Tomatoes** and **IMDB**. The latter was further divided into two subsets, namely **IMDB_3** and **IMDB_5**, containing partially overlapping records.

The dataset sizes vary across sources, with **IMDB_3** containing 2933 records, **IMDB_5** 6704 records, **Rotten Tomatoes** 3051 records, and **Roger Ebert** 3438 records.

Table 1 summarize the attributes and value types available in each dataset.

IMDB_3	DType
ID	object
Title	object
Year	object
Rating	float64
Director	object
Creators	object
Cast	object
Genre	object
Duration	object
ContentRating	object
Summary	object

(a) IMDB_3 schema

Rotten Tomatoes	DType
ID	object
Title	object
Year	object
Rating	object
Director	object
Creators	object
Cast	object
Genre	object
Duration	object
ContentRating	object
Summary	object

(b) Rotten Tomatoes schema

Roger Ebert	DType
id	int64
movie_name	object
year	float64
directors	object
actors	object
critic_rating	float64
genre	object
pg_rating	object
duration	object

(c) Roger Ebert schema

IMDB_5	DType
id	int64
movie_name	object
year	int64
directors	object
actors	object
movie_rating	float64
genre	object
duration	object

(d) IMDB_5 schema

Table 1: Datasets schema

2.1 Analysis and Data Preprocessing

An initial exploratory analysis of the data sources, conducted via “DataBlist”¹, an external tool for data inspection and preparation, revealed several data quality issues, including missing values and duplicate records, as well as concerns related to source consistency and reliability.

Duplicate and Missing values Duplicate records, identified using combinations of *movie title* and *year*, or *movie title* and *director* as matching keys, were not a major issue and were immediately addressed due to their low frequency and uniformity across all attributes, except for the source-specific identifier.

A different situation arose for missing values. While the *Rotten Tomatoes* and *IMDB* datasets remained below a moderate threshold of 7% missing values, *Roger Ebert* showed substantially higher rates, reaching 26% for the *directors* attribute and 16% for the *actors* attribute.

Roger Ebert	% of missing values
id	0%
movie_name	0%
year	0.141%
directors	25.309%
actors	16.535%
critic_rating	0%
genre	5.849%
pg_rating	13.189%
duration	7.508%

Table 2: Percentage of missing values in the *Roger Ebert* dataset

Investigating the causes of missing values, it was observed that a significant portion of the missing data in the *Roger Ebert* dataset is due to the nature of the source itself. As a movie review platform managed by professional editors, certain metadata fields such as *pg_rating* and *duration* are often not explicitly provided and are therefore absent.

Regarding the *directors* attribute, missing values were likely caused by limitations in the original extraction process. Director information is inconsistently embedded within the review pages, often appearing in heterogeneous textual patterns such as *Directed By*, *Written and Directed By*, or *Director* sections.

Although the exact implementation of the original scraper is unknown, it was observed that review pages can be reliably accessed through a few URL structures that can be reconstructed from existing dataset attributes.

¹ <https://app.datablist.com/>

```

base_url = "https://www.rogerebert.com/reviews/"

# possible URL patterns:
# 1) title-year
# 2) title
# 3) great-movie-title-year

# examples:
# base_url/title-year
# base_url/title
# base_url/great-movie-title-year

```

Fig. 6: Possible URL patterns used to access review pages in the Roger Ebert dataset.

This observation enabled the development of a revised scraping strategy, which systematically generated review URLs and extracted structured information directly from the source pages, significantly reducing the missing rate for the **directors** attribute to 7.508%.

A similar approach was applied to the *cast* attribute; however, improvements were limited. In many cases, cast information is not explicitly structured in dedicated sections but is embedded within the review text, making reliable extraction more challenging and resulting in only a marginal reduction in missing values.

Table 3 shows the updated missing-value statistics for the **Roger Ebert** dataset after the execution of the scraping process.

Roger Ebert	% of missing values
id	0%
movie_name	0%
year	0.141%
directors	7.508%
actors	14.398%
critic_rating	0%
genre	5.849%
pg_rating	13.189%
duration	7.508%

Table 3: Updated **Roger Ebert** statistics after the scraping process.

Source Reliability Further analysis showed how some sources exhibited characteristics that could potentially bias the integration process.

In particular, the Rotten Tomatoes dataset was observed to contain plot descriptions originating from external sources, including IMDb itself. Several of these descriptions also appeared to be truncated or incomplete. These observations raised concerns regarding the quality, consistency and independence of the extracted data.

In addition, the two IMDb datasets were also found to share a subset of records, indicating partial overlap between the corresponding sources.

These observations motivated the introduction of source-specific reliability weights during the data fusion phase, allowing the contribution of each source to be adjusted according to its perceived quality, completeness, and independence.

```
weights = {
    'IMDb_1' : 1.0,
    'Rotten Tomatoes': 0.2,
    'IMDb_2': 0.4,
    'Roger Ebert': 1.0
}
```

Fig. 7: Reliability weights assigned to each data source

2.2 Schema Alignment Configuration

As anticipated in the previous section, the schema alignment procedure relied on two implementations: a classical algorithmic method and an LLM-assisted one.

Traditional baseline The algorithm constructs a profile for each attribute, i.e., an internally generated representation of its descriptive features. Its performance is determined by the weighted similarity function 1, configured as follows:

$$(w_1, w_2, w_3, w_4) = (0.4, 0.3, 0.15, 0.15)$$

where w_1 , w_2 , w_3 , and w_4 correspond respectively to average attribute *length*, *word count*, *numeric density*, and *cardinality*. These weights reflect the higher discriminative power of textual structural features in distinguishing heterogeneous attributes such as **Cast** (long lists of names) versus **Director** or **Title** (shorter textual values).

Numerical attributes, such as **year** and **duration**, show significant differences in terms of *value range* and *average magnitude*, as captured by Function 2. In the experimental configuration, these components are equally weighted ($v_1 = 0.5$, $v_2 = 0.5$), reflecting their comparable discriminative power.

At a higher level, when numerical attributes are involved in the comparison, numerical similarity is weighted more heavily than structural similarity in the overall data similarity score, leading to the choice of $\alpha = 0.6$ in Function 3.

The final similarity score assigns greater importance to data similarity than to lexical similarity, reducing the likelihood of false positive matches caused by similar attribute names across datasets, such as **movie_rating**, **pg_rating**, and **critic_rating**. Consequently, the matching process defined in Function 4 prioritizes structural consistency over attribute-name

similarity ($\beta = 0.6$).

In the subsequent clustering phase, a similarity threshold of $\tau = 0.7$ was adopted to determine whether two attributes should be grouped together. This value was empirically selected to be consistent with the previously defined weighting scheme and to balance precision and recall in the alignment process.

LLM-assisted Alignment The LLM-assisted schema alignment relies on two prompt configurations, evaluated with three different models/systems: the standalone LLMs `meta-llama/llama-4-scout-17b-16e-instruct` and `openai/gpt-oss-120b`, and the integrated LLM system `groq/compound-mini`².

The first prompt is a generic, rule-based formulation suitable for any movie dataset.

```
"content": f"""
...
In this data integration pipeline, you are a data engineer
responsible for schema alignment...

Below are dataset names, sample excerpts from each dataset...

Rules:
- An attribute does not necessarily have a match in every dataset.
- If an attribute has no corresponding attribute in another dataset,
  leave the corresponding cell empty.
...

Output format example:
{output_header}
{example_row_1}
{example_row_2}

Datasets:
{datasets_text}

"""
```

Fig. 8: Generic prompt snippet used for LLM-assisted schema alignment

The second, more specific prompt, builds upon the first one by incorporating a manually crafted attribute metadata file that describes the role of each attribute within the dataset schema.

² <https://console.groq.com/docs/compound/systems/compound-mini>

An example of such a file is shown below:

```
{
  "attributes": [
    {
      "dataset": {
        "name": "imdb_1",
        "attributes_description": {
          "id": "unique film identifier",
          "title": "film's title",
          "year": "film's release year",
          "rating": "average IMDb user rating",
          "director": "film's directors",
          "creator": "film's creator",
          "cast": "film's cast",
          "genre": "film's genres",
          "duration": "film's runtime",
          "contentRating": "Motion picture content rating"
        }
      }
    },
    ...
  ]
}
```

Fig. 9: Attribute description json example

An additional control parameter, **temperature**, is used to regulate the LLM's level of randomness, thereby modulating the diversity of the generated outputs: higher temperature values increase output variability, which may improve performance in some settings but can also reduce determinism and consistency. In the reported experiments, the temperature was set to 0 in order to ensure deterministic and reproducible outputs.

2.2.1 Alignment Evaluation Ground truth

Since no reference schema mappings were available for the selected datasets, the ground truth was created by manually identifying the correct semantic correspondences between the attributes of the different sources.

The resulting ground truth consists of 12 semantic attribute groups. Each group was converted into all possible pairwise attribute correspondences (e.g., **A.title**–**B.title**, **A.title**–**C.title**), yielding a total of 49 pairwise links used to compute precision, recall, and F1-score.

2.3 Record Linkage Configuration

The following paragraphs report the parameter values adopted for the blocking, matching, and clustering stages of the record linkage pipeline.

Blocking The selection of the blocking keys was primarily driven by the objective of reducing the number of candidate comparisons while preserving the ability to group potentially matching records.

This choice was also influenced by the characteristics of the datasets. Attributes containing a large number of missing values, such as **Cast**, proved unsuitable as blocking keys because many records cannot be reliably assigned to a canopy, leading to additional clustering iterations and a larger number of candidate groups. Conversely, attributes such as **Year** and **Genre** are too coarse-grained and tend to generate overly large canopies, thereby reducing the effectiveness of the blocking stage.

For these reasons, the selected blocking keys are **Title** and **Director**, as they exhibit high discriminative power and relatively low redundancy across records. They are encoded using character-level bigram ($x = 2$) and trigram ($y = 3$) in the TF-IDF representations shown in Figure 3.

This choice is motivated by the need to capture partial similarities between differently formatted strings while avoiding the natural sparsity of the attributes in the datasets.

Traditional Record Matching The record matching strategy described in Section 1.2.2 was instantiated according to the characteristics of the considered movie datasets. Based on the schema alignment results, four attributes were selected as matching features: **Title**, **Year**, **Director**, and **Cast**. These attributes were chosen because they provide complementary evidence for identifying the same movie entity across heterogeneous sources.

The record-level similarity function was configured using the following attribute weights:

$$w_t = 0.50, \quad w_y = 0.25, \quad w_d = 0.15, \quad w_c = 0.10$$

The **Title** attribute was assigned the highest weight since it represents the main identifying information of a movie entity. The **Year** attribute was given a lower but still significant contribution because it helps distinguish movies with similar titles, such as remakes or different adaptations. The **Director** and **Cast** attributes were considered additional semantic signals to improve matching reliability.

The similarity function associated with each attribute is reported in Table 4.

Attribute	Similarity Function	Weight
Title	Text similarity	0.50
Year	Rule-based comparison	0.25
Director	Hybrid similarity	0.15
Cast	Jaccard similarity	0.10

Table 4: Record matching configuration.

The **Title** attribute was compared using a text similarity function based on the character-level **ratio** metric provided by the RapidFuzz library.

The **Year** attribute was configured using a rule-based comparison. Records with identical years receive maximum similarity:

$$S_{year} = 1$$

A difference of one year was considered a weak indication of compatibility and was assigned a lower similarity score:

$$S_{year} = 0.25$$

Differences larger than one year were considered incompatible:

$$S_{year} = 0$$

The **Director** attribute was configured using the hybrid similarity strategy, since the analysis of the datasets showed heterogeneous representations of this attribute. In particular, some records contained a single director value, while others included multiple directors, requiring a comparison strategy able to handle both representations.

The **Cast** attribute was configured using Jaccard similarity, as actor information is represented as a collection of values.

For all considered attributes, missing values were excluded from the similarity aggregation, and the final score was normalized over the available attributes.

The final matching decision was obtained by applying the decision threshold defined in Equation 9. In the experiments, the selected threshold was set to $\theta = 0.75$. Candidate pairs satisfying this condition were classified as matches and passed to the subsequent entity clustering phase.

LLM-assisted Record Matching The LLM-assisted strategy described in Section 1.2.3 was configured by setting the candidate threshold and the acceptance criteria used during the experiments.

The relaxed candidate threshold was set to:

$$\theta' = 0.65$$

Candidate pairs with a similarity score below this value were discarded, while pairs satisfying the standard matching threshold $\theta = 0.75$ were directly accepted without LLM evaluation. Pairs within the ambiguous range were instead forwarded to the LLM:

$$0.65 \leq S_{record}(r_i, r_j) < 0.75$$

For each ambiguous pair, the LLM received the values of the matching attributes used in the record comparison, namely **Title**, **Year**, **Director**, and **Cast**. The expected response format consisted of a match indicator, a confidence score in the range $[0, 1]$, and a short explanation.

The LLM evaluation was performed through the Groq API using the `llama-3.3-70b-versatile` and `openai/gpt-oss-120b` models. The temperature parameter was set to 0 in order to obtain deterministic responses across repeated evaluations.

The confidence threshold for accepting LLM-based matches was set to:

$$\gamma = 0.75$$

Entity Clustering Configuration Unlike the previous stages, the entity clustering procedure does not require the tuning of algorithm-specific hyperparameters. Entity clusters are obtained by computing the connected components of the graph generated from the accepted record matches. Consequently, all experiments employ the same deterministic configuration, and the resulting clusters depend exclusively on the output of the record matching stage.

2.3.1 Record Matching Evaluation Ground Truth

The record matching phase was evaluated using the ground truth annotations provided by the Magellan framework, which covers only two dataset pairs and is divided into two independent evaluation scenarios: *Movies 3*, corresponding to the IMDb_3–Rotten Tomatoes pair, and *Movies 5*, corresponding to the IMDb_5–Roger Ebert pair.

To ensure consistency between the reference annotations and the final integrated data, the ground truth annotations were filtered to retain only pairs whose records were still present after the preprocessing and normalization stages.

In particular, for *Movies 3*, the filtering process retained 389 ground truth pairs out of 399 annotated pairs. One entry without a valid gold label was discarded, while 9 pairs were removed because the corresponding records were not present in the final dataset (4 true matches and 5 non-matches). For *Movies 5*, 361 pairs were retained out of 373 annotated pairs, with 12 discarded (2 true matches and 10 non-matches).

2.4 Data Fusion

The data fusion stage did not require additional parameter tuning. The procedure is fully deterministic and is applied uniformly to all clusters produced by the record linkage phase, aggregating attribute values according to the strategy described in Section 1.3.

3 Results

3.1 Schema Alignment

The schema alignment phase was evaluated by comparing the predicted attribute correspondences against the manually constructed ground truth.

Both the traditional profile-based and the LLM-assisted approach predicted 49 pairwise correspondences, matching the number of links in the ground truth.

Metric	Value
Alignment results	
Ground truth pairwise links	49
Predicted pairwise links	49
True positives	47
False positives	2
False negatives	2
Evaluation metrics	
Precision	0.959
Recall	0.959
F1-score	0.959

(a) Traditional profile-based alignment

Metric	Value
Alignment results	
Ground truth pairwise links	49
Predicted pairwise links	49
True positives	49
False positives	3
False negatives	0
Evaluation metrics	
Precision	0.942
Recall	1.000
F1-score	0.970

(b) Generic prompt

Metric	Value
Alignment results	
Ground truth pairwise links	49
Predicted pairwise links	52
True positives	49
False positives	0
False negatives	0
Evaluation metrics	
Precision	1.000
Recall	1.000
F1-score	1.000

(c) Enhanced prompt

Table 5: Schema alignment performance comparison across configurations.

As can be seen, the traditional profile-based alignment correctly identified 47 correspondences, while two were incorrectly matched, resulting in two false positives and two false negatives. Overall, the method achieved a precision, recall, and F1-score of 0.959.

Among the LLMs evaluated in the experimental setup, `openai/gpt-oss-120b` was selected for the subsequent experiments. While the generic prompt produced comparable performance across the evaluated models, the introduction of attribute-level metadata highlighted substantial differences. In particular, `openai/gpt-oss-120b` successfully exploited the additional contextual information, whereas `meta-llama/llama-4-scout-17b-16e-instruct` exhibited a noticeable degradation in alignment performance. `groq/compound-mini` achieved comparable results to `openai/gpt-oss-120b`, but with substantially higher token consumption. Therefore, all results reported below refer to `openai/gpt-oss-120b`.

3.2 Record Linkage

Blocking As described in Section 2.3, the blocking phase was performed using the Canopy clustering approach to reduce the search space while preserving potentially matching records.

The input dataset contained 16126 records, which were assigned to 11991 canopy blocks. Since Canopy clustering allows overlapping blocks, some records were assigned to multiple canopies, resulting in a total of 5994 records appearing in more than one canopy. All possible record pairs within each canopy block were then generated, resulting in 30279 candidate pairs, used as input for the subsequent record matching phase.

Record Matching Both for the traditional baseline and the LLM-assisted approach, the evaluation was performed separately on the two available ground truth scenarios.

The similarity-based matcher evaluates candidate pairs by combining attribute-level similarities over **Title**, **Year**, **Director**, and **Cast** information. Out of the 30279 candidate pairs generated during the blocking phase, 5036 pairs were classified as matches.

Table 6.a reports the statistics for the 2038 predicted matches in the Movies 3 scenario, corresponding to the IMDb₃–Rotten Tomatoes dataset pair. Among these, 149 pairs are present in the validated ground truth set, which contains 165 true matching pairs.

The similarity-based matcher achieved perfect precision, indicating that no false positives were produced within the labeled ground truth. However, a recall of 0.903 highlights that a subset of true correspondences was not retrieved, mainly due to differences and incompleteness in the available metadata across the two sources.

Table 6.b reports the statistics for the Movies 5 scenario, corresponding to the Roger Ebert–IMDb₅ dataset pair. In this case, 204 predicted matches are contained in the validated ground truth set, which includes 239 annotated true matches.

The matcher maintains very high precision (0.995), with only a single false positive within the labeled ground truth. However, recall decreases to 0.849, indicating a larger proportion of missed correspondences compared to Movies 3. This suggests that the Roger Ebert–IMDb₅ pair is more challenging, likely due to higher variability and incompleteness in the available metadata.

Metric	Value
Alignment results	
Predicted matches	2038
True matches in ground truth	165
Predicted pairs inside ground truth	154
True positives	154
False positives	0
False negatives	11
Evaluation metrics	
Precision	1.000
Recall	0.933
F1-score	0.966

(a) Movies 3

Metric	Value
Alignment results	
Predicted matches	276
True matches in ground truth	239
Predicted pairs inside ground truth	205
True positives	204
False positives	1
False negatives	35
Evaluation metrics	
Precision	0.995
Recall	0.853
F1-score	0.919

(b) Movies 5

Table 6: Traditional Similarity-based record matching performance.

The LLM-assisted approach was evaluated using two different language models: `openai/gpt-oss-120b` and `llama-3.3-70b-versatile`.

For Movies 3, both models achieved the same number of true positives and identical evaluation metrics, while producing a slightly different number of predicted matches, as reported in Table 7.

Metric	gpt-oss-120b	llama-3.3-70b-versatile
Matching results		
Predicted matches	2159	2136
True matches in ground truth	165	165
True positives	155	155
False positives	0	0
False negatives	10	10
Evaluation metrics		
Precision	1.000	1.000
Recall	0.939	0.939
F1-score	0.969	0.969

Table 7: LLM-assisted record matching performance on Movies 3.

Both LLMs achieved the same matching performance, with perfect precision and identical recall and F1-score values.

Compared with the classical similarity-based matcher, both models correctly identified five additional true matches, improving recall from 0.903 to 0.933 while maintaining no false

positives. This indicates that the LLM-based approach was able to recover additional correspondences without reducing the matching accuracy.

For Movies 5, the LLM-assisted approach produced 304 predicted matches using `openai/gpt-oss-120b` and 297 using `llama-3.3-70b-versatile`. Among these predictions, the two models identified 226 and 221 true matches respectively from the validated ground truth set, which contains 239 annotated true matches, as reported in Table 8.

Metric	gpt-oss-120b	llama-3.3-70b-versatile
Matching results		
Predicted matches	304	297
True matches in ground truth	239	239
True positives	226	220
False positives	1	1
False negatives	13	19
Evaluation metrics		
Precision	0.996	0.995
Recall	0.946	0.920
F1-score	0.970	0.957

Table 8: LLM-assisted record matching performance on Movies 5.

The results here show a larger improvement compared with the classical similarity-based matcher. The `openai/gpt-oss-120b` model correctly identified 23 additional true matches, while `llama-3.3-70b-versatile` identified 18 additional true matches. Both models maintained very high precision, while substantially improving recall from 0.849 in the classical approach to 0.946 and 0.925, respectively. This indicates that the LLM-assisted approach is particularly effective in more challenging matching scenarios where metadata variability makes similarity-based matching less reliable.

Entity Clustering The traditional similarity-based approach produced 2628 multi-record clusters, while 9694 records remained as singletons. Since LLM-assisted matchers identified a larger number of record correspondences, the clustering phase generated a higher number of clusters containing two or more records.

Specifically, the `openai/gpt-oss-120b` model produced 2793 clusters, whereas `llama-3.3-70b-versatile` produced 2740.

3.3 Data Fusion

As expected, the data fusion phase produced the same number of final records as the number of multi-record clusters obtained during the entity clustering phase, since a single fused record was obtained from each cluster containing two or more records.

3.4 Final Dataset

Overall, the traditional similarity-based approach produced a final dataset containing 12322 entities, while the LLM-assisted approaches produced 12107 entities for `openai/gpt-oss-120b` and 12180 entities for `llama-3.3-70b-versatile`.

These results are consistent with previous stages, where the LLM-based methods identified a higher number of matches, leading to a larger number of clusters and, consequently, fused entities.

4 Error Analysis

This section analyzes the most representative errors produced throughout the integration pipeline. Examining both successful and unsuccessful predictions provides a deeper understanding of the observed quantitative results, highlighting the limitations of the traditional profile-based approach, the contribution of Large Language Models, and the impact of incorporating attribute-level metadata into the alignment process.

4.1 Schema Alignment

As observed in the previous section, the traditional profile-based schema alignment produced two false positives and two false negatives.

False Positive Links

- `imdb_3.contentrating` $\leftrightarrow$ `roger_ebert.critic_rating`
- `roger_ebert.critic_rating` $\leftrightarrow$ `rotten_tomatoes.contentrating`

Although both attributes contain the term *rating*, they represent different concepts. The `contentrating` attribute stores the parental classification of a movie (e.g., `PG`, `PG-13`, or `R`), whereas `critic_rating` represents a numerical evaluation assigned by professional reviewers. Since the structural profiles of these attributes are not sufficiently discriminative, the lexical similarity biased the final score, leading the algorithm to incorrectly merge them into the same semantic group.

The use of LLMs improved this behaviour by providing a more semantic interpretation of attribute meanings; however, this also introduced a different type of false positive. In particular, the model associated `critic_rating` with `movie_rating` attributes, as both represent numerical evaluations related to a movie. Although this association is semantically plausible at a high level, the two attributes describe different concepts. The `critic_rating` attribute represents the score assigned by an individual professional reviewer in the Roger Ebert dataset, whereas the corresponding rating attributes in the other datasets refer to different notions, such as aggregated user ratings or content classifications. Without explicit attribute-level context, the model overestimated their semantic similarity, considering their shared association with movie evaluation as sufficient evidence for a match.

False Negative Links

- `imdb_3.contentrating` $\leftrightarrow$ `roger_ebert.pg_rating`
- `roger_ebert.pg_rating` $\leftrightarrow$ `rotten_tomatoes.contentrating`

These missed correspondences are closely related to the false positive involving `critic_rating`. Since the clustering procedure enforces the constraint that each cluster can contain

at most one attribute from each dataset, the incorrect association of `critic_rating` with `concentrating` prevented the semantically equivalent `pg_rating` attribute from being assigned to the same cluster. As a result, correct correspondences were not recovered during the final alignment stage.

This issue was completely resolved by the LLM-assisted approach. By leveraging semantic understanding, the model correctly identified that `concentrating` and `pg_rating` represent the same attribute concept despite their different naming conventions, while avoiding the incorrect association with `critic_rating` when additional attribute context was provided.

4.2 Record Matching

The record matching phase exhibited a different type of behavior. Overall, the system achieved extremely high precision, with only one false positive observed in the Movies 5 evaluation. Additionally, a representative false negative case was analyzed to investigate the limitations of the similarity-based matching strategy and understand the causes behind missed matches.

False Positive Match The identified false positive corresponds to the following pair of records:

Attribute	c688	d1232
Title	Afflicted	afflicted
Year	2013	2014
Director	Derek Lee	Clif Prowse, Derek Lee
Cast	Clif Prowse, Derek Lee, Michael Gill	Baya Rehaz, Derek Lee, Clif Prowse
Genre	Horror, Mystery, Thriller	Action, Thriller
Duration	85	85

Table 9: False positive record pair (`d1232` $\leftrightarrow$ `c688`).

Although the two records correspond to different entities according to the ground truth, they exhibit a high degree of similarity. The titles differ only in capitalization, the duration is identical, the director information partially overlaps, and two cast members (*Derek Lee* and *Clif Prowse*) appear in both records. Furthermore, the release years differ by only one year, which is considered partially compatible by the rule-based year similarity function.

Consequently, the weighted combination of these similarities produced an overall similarity score of 0.763, exceeding the matching threshold of 0.75. Therefore, the two records were incorrectly classified as a match. Since record pairs above this threshold are directly accepted by the traditional matcher and are not forwarded to the LLM verification step, this false positive is necessarily inherited by the LLM-based pipeline as well.

This example illustrates a limitation of similarity-based record linkage: when several highly discriminative attributes simultaneously exhibit strong agreement, their combined contribution may outweigh the remaining differences, leading distinct real-world entities to be incorrectly merged.

False Negative Match A false negative case was identified for the following pair of records:

Attribute	a48	b50
Title	Beauty and the Least: The Misadventures of Ben Banks	Ben Banks
Year	2013	<NA>
Director	Bryce Clark	Bryce Clark
Cast	Ben Banks, Katharine Towne, Mischa Barton	Mischa Barton, Ben Banks, Alex Beh, Katharine Towne
Genre	Comedy, Drama, Romance	Drama, Comedy
Duration	85	90

Table 10: False negative record pair (b50 $\leftrightarrow$ a48).

Although the two records refer to the same movie, they were not classified as a match by the record matching component. A detailed analysis of this specific record pair was performed by recomputing the attribute-level similarities and the corresponding weighted score. The resulting similarity score was 0.497, which is significantly below the matching threshold of 0.75.

The main reason for this error is the strong influence of the title attribute, which has the highest weight in the matching configuration (0.50). The two sources provide different representations of the same title: the first record contains the complete title (*Beauty and the Least: The Misadventures of Ben Banks*), whereas the second one uses a shortened version (*Ben Banks*). As a consequence, the title similarity was only 0.295, substantially reducing the final weighted score.

However, the remaining attributes provided strong evidence that the records correspond to the same entity. In particular, the director matched exactly, obtaining a similarity score of 1.0, while the cast attribute achieved a similarity of 0.75 due to the large overlap between the two lists of actors.

This example highlights the limitations of assigning a dominant weight to a single attribute in record matching. While emphasizing the title generally improves precision, it can negatively affect recall when different data sources contain alternative title representations, abbreviations, or extended versions. Possible improvements include applying more robust title normalization techniques, introducing partial title matching strategies, or slightly reducing the influence of the title attribute to allow supporting evidence from other attributes to contribute more effectively.

5 Discussion

5.1 Role of LLMs in Big Data Integration

The experimental results suggest that the main contribution of Large Language Models in Big Data Integration lies in their ability to capture semantic relationships that are difficult to identify through purely deterministic approaches. In particular, both stages where LLMs were applied showed improved results compared to traditional methods, demonstrating their effectiveness in resolving ambiguous cases and handling heterogeneous representations.

However, the experiments also highlight that semantic reasoning alone is not sufficient to guarantee correct results. In schema alignment, for example, LLMs may generate plausible but incorrect correspondences when provided with limited context, as observed in the association between `critic_rating` and movie rating attributes. This shows the importance of providing additional metadata and constraints to guide the model towards more accurate decisions.

Overall, LLMs should be considered as a complementary component rather than a replacement for traditional approaches. Deterministic methods remain essential for tasks requiring efficiency, reproducibility, and strict constraints, while LLMs provide additional value in scenarios where semantic understanding and contextual interpretation are required.

5.2 Practical Considerations: Cost, Latency, and Reproducibility

The integration of LLM-based components introduces additional practical considerations compared to fully deterministic approaches, mainly related to computational cost, response latency, and reproducibility of results.

In schema alignment, the main trade-off concerns the amount of contextual information provided to the model. Enriching prompts with additional metadata, such as attribute descriptions, improves the ability of the LLM to understand attribute semantics and generate more accurate mappings. However, this additional context increases the prompt size and, consequently, the number of processed tokens, resulting in higher costs and longer response times. Therefore, the amount of contextual information provided to the model should be carefully balanced according to the desired level of accuracy and available resources.

Regarding record matching, computational costs become particularly relevant due to the potentially large number of candidate record pairs requiring evaluation. Forwarding every ambiguous pair to the LLM would lead to a significant increase in API calls, latency, and token consumption. To mitigate this issue, a caching mechanism was introduced in the pipeline to avoid redundant evaluations. This reduces unnecessary costs while preserving consistency across repeated executions.

The selection of the similarity range forwarded to the LLM also represents an important trade-off. Increasing the interval of similarity scores considered as ambiguous allows the

model to evaluate a larger number of uncertain cases, potentially improving recall by recovering additional matches. However, this comes at the cost of additional LLM queries and higher resource consumption, and it does not necessarily guarantee improved precision. Therefore, the choice of the candidate threshold must balance matching performance with computational efficiency.

Finally, although LLM-based components provide valuable semantic reasoning capabilities, their integration requires careful management of experimental conditions to improve reproducibility. In this work, deterministic settings were adopted by fixing the temperature parameter to 0, reducing variability across repeated executions.

Conclusion

This work presented a data integration pipeline for heterogeneous movie datasets, addressing schema alignment, record linkage, entity clustering, and data fusion through a combination of traditional methods and Large Language Models.

The experimental results showed that traditional techniques provide strong and reliable performance, while LLM-based components improve the resolution of ambiguous cases by capturing semantic relationships that are difficult to identify through purely similarity-based approaches. However, their effectiveness depends on appropriate prompt design and sufficient contextual information.

Overall, the results indicate that LLMs represent a valuable complement to traditional data integration methods, enhancing their ability to handle heterogeneous and semantically complex scenarios.